

TECHNICAL AUDIT REPORT

DesignSystemMissionControl

Auditoria tecnica integral orientada a determinar el porcentaje real de finalizacion, la calidad de produccion y la brecha restante para alcanzar un estado de release confiable.

FECHA

2026-03-19

RESULTADO

73%

ESTADO

**Beta
avanzada**APTO PARA
PROD**No aun**

1. Resumen Ejecutivo

El proyecto presenta una base tecnica seria: arquitectura separada frontend/backend, esquema compartido con Drizzle y Zod, multiples modulos funcionales, build de produccion exitosa y una cobertura visual amplia del scope operativo. Sin embargo, el porcentaje visible de interfaz sobrestima el grado real de terminacion.

El sistema esta mas cerca de un entorno de **staging operativo** que de una plataforma endurecida para produccion. La mayor parte de los CRUD nucleares existe, pero persisten inconsistencias de autenticacion, modulos parcialmente simulados, deuda tecnica documentada en el propio codigo y decisiones de seguridad que todavia no son aceptables para un despliegue productivo estricto.

FUNCIONALIDAD

71%

Amplia superficie implementada, pero con flujos incompletos o engañosos.

PRACTICIDAD
UX**78%**

Buena jerarquia visual; manejo de errores y acciones rapidas todavia irregular.

LIMPIEZA

74%

Buen uso de shared schema y UI reuse, con varias excepciones y atajos temporales.

SEGURIDAD

61%

Base razonable, pero aun con huecos que bloquean el sello de produccion.

2. Alcance y Metodologia

La auditoria se realizo mediante inspeccion cruzada del proyecto local, con enfasis en cuatro ejes:

- 1. Correspondencia entre rutas del backend en `server/routes.ts`, controladores en `server/controllers/` y rutas del frontend definidas en `client/src/App.tsx`.
- 2. Revision de paginas y componentes clave para identificar dead ends, placeholders, estados de carga y rutas engañosas.
- 3. Analisis de arquitectura compartida, schema reuse, deuda tecnica documentada y patrones redundantes.
- 4. Revision de seguridad operacional: autentificacion, exposicion de datos, sanitizacion y endurecimiento de respuestas API.

Verificacion tecnica realizada: `npm run build` completo con exito, lo que confirma compilacion y bundling productivo. La suite `npm test` no pudo ejecutarse de forma concluyente en este entorno porque Vitest fallo al arrancar con `spawn EPERM` al cargar `vitest.config.ts`. Por lo tanto, no se obtuvo una lectura util del estado real de los tests automatizados.

3. Porcentaje de Finalizacion Real

SUBMODULO	ESTIMACION	JUSTIFICACION
CRM / Leads	85%	CRUD consistente, metricas y conversion de leads presentes. Buen grado de integracion con frontend.
Clientes	85%	Listado, detalle y tabs asociados implementados. Debilidad: el detalle consume toda la coleccion para resolver un registro.
Proyectos	82%	Lista, detalle, control, entregables, adjuntos y calculo financiero existen. Persisten atajos temporales en upload.
Finanzas	76%	Transacciones y resumen presentes, pero el calendario de pagos no es un ledger real sino una derivacion heuristica desde proyectos.
Equipo / Personal	84%	Modulo bien conectado con roles, asignaciones y formularios; hay exposicion

SUBMODULO	ESTIMACION	JUSTIFICACION
		excesiva de detalles internos en errores.
Servicios / Proveedores / POES / Usuarios / Actividad	84%	CRUD y vistas operativas maduras en terminos relativos.
Ads	45%	UI existente, pero accesibilidad y autenticacion inconsistentes; OAuth no implementado y flujos clave siguen simulados.
Settings	40%	Funciona como singleton sobre un usuario admin por defecto, no como configuracion multiusuario endurecida.
Profile	35%	Avatar funcional; el resto del perfil esta explicitamente pendiente de implementacion.

4. Diagnostico por Pilar

4.1 Funcionalidad

La arquitectura de enrutado del backend esta organizada como un ensamblado de routers. En `server/routes.ts` se montan routers para auth, campaigns, clients, team, resources, financial, projects, ads, settings, contacts, billing profiles, digital assets, installments, services, suppliers, leads, poes, users, project team y audit logs.

En el frontend, `client/src/App.tsx` define un arbol de paginas renderizables amplio: dashboard, clientes, detalle de cliente, proyectos, detalle de proyecto, recursos, equipo, KPIs, finanzas, perfil, settings, CRM, POES, calendario de pagos, control de proyectos, usuarios, proveedores, servicios y actividad.

El problema no es ausencia de archivos sino diferencias entre superficie visible y flujo productivo real. Algunos ejemplos relevantes:

- **Ads no esta realmente expuesto como modulo principal:** `/ads` y `/ads/command-center` redirigen a `/crm`, aunque existe una pagina `ads-command-center` y un acceso a `/ads/settings`.
- **Quick Create contiene un dead end:** "Nueva Nota" abre un modal cuyo contenido es solo "Modulo de notas en desarrollo".
- **Perfil incompleto:** el propio archivo indica que los cambios distintos al avatar estan pendientes de implementacion.
- **Command Palette incompleta:** lista campañas activas, pero el `onSelect` solo cierra el dialogo y no navega ni ejecuta una accion.

EVIDENCIA	LECTURA	IMPACTO
<code>client/src/App.tsx:99-100</code>	<code>/ads</code> y <code>/ads/command-center</code> redirigen a CRM	Modulo Ads visible pero no accesible como destino principal
<code>client/src/components/quick-create-menu.tsx:115-119</code>	Placeholder en "Nueva Nota"	Dead end funcional
<code>client/src/pages/profile.tsx:186-193</code>	Perfil no implementado fuera del avatar	Superficie funcional inflada
<code>client/src/components/app-layout.tsx:298-305</code>	Command palette sin accion real	UX aparenta potencia, pero no entrega utilidad operativa completa

4.2 Practicidad y UX

La capa visual esta bien trabajada y coherente con la estetica "Ethereal Stealth". La jerarquia de informacion de `dashboard`, `control-proyectos` y `client-detail` es superior al promedio: buena densidad, tarjetas claras, paneles de control y layout reutilizable.

Tambien existe una base de estados de carga razonable: `LoadingFallback` global, spinners locales y algunos empty states. No obstante, el manejo de error esta menos maduro:

- El `ErrorBoundary` global muestra error y `componentStack` completos al usuario final.
- No todas las paginas tienen fallbacks dedicados o mensajes de recuperacion consistentes.
- Varios flujos hacen `fetch` directo y, cuando fallan por auth, el comportamiento puede sentirse erratico.

Observacion UX-pragmatica: la app luce mas terminada de lo que realmente esta. Este tipo de desalineacion es peligrosa porque genera falsa confianza operativa: el usuario puede asumir que un modulo esta listo por la calidad visual, aunque por debajo todavia dependa de placeholders o integraciones parciales.

4.3 Limpieza y Arquitectura

Hay una decision acertada en el uso de `shared/schema.ts` como fuente de tipos y validaciones, complementado por una libreria UI reutilizable en `client/src/components/ui/`. Esto reduce drift de contratos y facilita coherencia entre formularios y controladores.

El deterioro aparece en los bordes:

- Coexisten un wrapper autenticado robusto en `client/src/lib/api.ts` y multiples llamadas `fetch` crudas que lo evitan.
- `client/src/pages/client-detail.tsx` resuelve un cliente puntual descargando todos los clientes y filtrando en memoria.
- `client/src/pages/project-detail.tsx` usa un data URL como "for now" para archivos, en lugar de un flujo de storage productivo.
- Hay deuda tecnica explicitamente documentada en torno a `webhookUrl` y al controlador de settings.
- La ruta `/proyectos-test` sigue expuesta en el router productivo.

ELEMENTO	ESTADO	COMENTARIO ARQUITECTONICO
Schema compartido	Bueno	Reutilizacion consistente de tipos y Zod en varios modulos.
UI reusable	Bueno	La carpeta <code>components/ui</code> soporta una base visual consistente.
Disciplina de acceso API	Inconsistente	Existen dos patrones de acceso: uno seguro y otro ad hoc.
Deuda tecnica documentada	Visible	El propio codigo reconoce features no implementadas o provisionales.

4.4 Seguridad y Hardening

El sistema ya tiene piezas correctas: `requireAuth` exige Bearer token valido, el secreto JWT se valida por longitud minima y los schemas usan transformaciones para filtrar entrada. Sin embargo, todavia hay varios riesgos incompatibles con un entorno de produccion serio.

Conclusion de seguridad: el proyecto no esta listo para considerarse "hardened". La seguridad actual es suficiente para una beta privada, pero insuficiente para una exposicion amplia o para manejar datos operativos sensibles con garantias altas.

- **Auth inconsistente en frontend:** varias pantallas protegidas no usan el wrapper que inyecta JWT.
- **Settings singleton inseguro:** el controlador manipula el usuario `admin` por defecto y expone `apiKey`, `username` y `role`.
- **Refresh tokens en texto claro:** el campo se llama `tokenHash`, pero el valor se guarda y consulta sin hash real.
- **Exposicion excesiva de errores:** algunos endpoints devuelven `details` y `stack` al cliente.

- **Saneamiento parcial:** `sanitizeString` ayuda, pero no reemplaza sanitizacion contextual de salida ni una politica uniforme de render seguro.

5. Critical Red Flags

SEVERIDAD	HALLAZGO	JUSTIFICACION	EVIDENCIA
Critica	Rutas protegidas fallables por auth inconsistente	El backend exige Authorization: Bearer, pero paginas como Ads y Settings hacen fetch directo sin pasar por el wrapper autenticado. En uso real esto puede traducirse en 401 silenciosos o UX rota.	<code>client/src/lib/api.ts:52-67</code> <code>client/src/pages/ads-settings.tsx:95-118,131-171</code> <code>client/src/pages/ads-command-center.tsx:60-85</code> <code>client/src/pages/settings.tsx:64-83,108-112</code>
Critica	Configuracion global montada sobre usuario admin por defecto	El controlador de settings no usa el usuario autenticado; opera sobre un singleton <code>admin</code> , creando incluso credenciales por defecto si no existen. Esto rompe aislamiento multiusuario y es una mala practica operativa y de seguridad.	<code>server/controllers/settings.ts:21-36</code> <code>server/controllers/settings.ts:39-60</code> <code>server/controllers/settings.ts:67-109</code>

SEVERIDAD	HALLAZGO	JUSTIFICACION	EVIDENCIA
Critica	Refresh tokens almacenados en texto claro	El nombre tokenHash induce una proteccion inexistente. El token generado se guarda tal cual y se consulta por igualdad exacta. Si la base se expone, el refresh token queda reutilizable.	server/controllers/auth.ts:38-49 server/storage.ts:552-560
Alta	Ads no esta terminado aunque la UI lo sugiera	OAuth devuelve 501, request review sigue mockeado, y la navegacion principal a Ads dirige a CRM. Es el principal caso de completion inflation del proyecto.	client/src/App.tsx:99-101 client/src/pages/ads-command-center.tsx:136-139 server/controllers/ads.ts:156-163 server/controllers/ads.ts:333-352
Alta	Exposicion de stack traces y detalles internos	Algunos endpoints devuelven stack y mensajes internos al cliente. El ErrorBoundary global tambien expone componentStack. Esto amplia	server/controllers/financial.ts:112-116 server/controllers/team.ts:40-43 client/src/components/error-boundary.tsx

SEVERIDAD	HALLAZGO	JUSTIFICACION	EVIDENCIA
		superficie de fuga tecnica.	
Media	Dead ends visibles al usuario	Hay acciones que parecen listas pero no entregan resultado operativo: "Nueva Nota", palette sin accion y perfil parcialmente deshabilitado.	client/src/components/quick-create-menu.tsx:115-119 client/src/components/app-layout.tsx:298-305 client/src/pages/profile.tsx:186-193
Media	Calendario financiero derivado y no contable	La pagina payment-calendar calcula pagos a partir de campos del proyecto en memoria, aunque el backend ya expone endpoints de parcialidades. El modulo sirve como visualizacion, no como fuente contable.	client/src/pages/payment-calendar.tsx:49-98 server/controllers/installments.ts:9-39

6. Evidencia de Correspondencia Ruta-Controlador-Pagina

La cobertura base existe. El problema dominante no es "ruta sin archivo", sino "ruta accesible con integracion parcial o política de acceso defectuosa".

AREA	FRONTEND	BACKEND	ESTADO
Clientes	/clientes, /clientes/:id	server/controllers/clients.ts + tabs auxiliares	Implementado con reservas
Proyectos	/proyectos, /proyectos/:id, /control- proyectos	server/controllers/projects.ts, installments.ts, project-team.ts	Implementado
Finanzas	/finanzas, /calendario-pagos	server/controllers/financial.ts, installments.ts	Parcialmente confiable
Equipo	/equipo	server/controllers/team.ts, agency.ts	Implementado
CRM	/crm	server/controllers/leads.ts	Implementado
Ads	/ads/settings, UI para command center no principal	server/controllers/ads.ts	Incompleto
Profile	/profile	/api/auth/me, /api/users/me/avatar	Parcial
Settings	/settings	server/controllers/settings.ts	Funciona, pero con diseño inseguro

7. Hallazgos Detallados por Dominio

7.1 CRM y Clientes

Es uno de los dominios mejor resueltos del proyecto. El backend de clientes en `server/controllers/clients.ts` tiene CRUD completo y el detalle de cliente agrega tabs para contactos, facturacion, assets y documentos. La debilidad principal es la eficiencia y pureza del acceso: `client-detail.tsx` resuelve un cliente individual descargando toda la lista y filtrando en memoria.

7.2 Proyectos

El dominio de proyectos es amplio y relativamente maduro. Existen lista, detalle, entregables, adjuntos, equipo, servicios, rentabilidad y parcialidades. La principal desviacion de produccion

es el flujo de archivo "for now" basado en data URLs, que no debería permanecer en una release endurecida.

7.3 Finanzas

El backend financiero esta razonablemente bien modelado para transacciones y resumen. La desviacion esta en la experiencia del calendario: en vez de usar el dominio de parcialidades como fuente primaria, la UI genera nodos de cobro desde proyectos y estados derivados. El modulo es util como visual operativo, pero no debería considerarse libro mayor.

7.4 Ads

Es el area mas incompleta del sistema. La UI y la cantidad de endpoints pueden inducir a pensar que el modulo esta avanzado, pero el analisis demuestra lo contrario: autenticacion inconsistente en frontend, OAuth no implementado, review requests mockeados y accesibilidad principal ausente por redirects.

7.5 Settings y Profile

Estos dos dominios concentran deuda silenciosa. Settings aparenta ser una configuracion madura, pero en realidad opera sobre un admin singleton por defecto. Profile luce completo, aunque solo el avatar tiene backend real integrado; el resto del formulario esta reconocido como pendiente.

8. Mapa de Pendientes para Llegar al 100%

1. Forzar que todo acceso autenticado use `request/apiRequest` y eliminar `fetch` crudo en pantallas protegidas.
2. Reescribir `server/controllers/settings.ts` para trabajar con el usuario autenticado, con controles por rol y sin usuario admin singleton.
3. Hashear refresh tokens en reposo y rotarlos correctamente.
4. Eliminar `stack` y `details` sensibles de respuestas productivas.
5. Completar Ads de verdad: OAuth, review notification real y navegacion principal sin redirects engañosos.
6. Terminar Profile o retirar los controles no operativos hasta implementarlos.
7. Vincular `payment-calendar` al dominio real de parcialidades o a un ledger financiero persistido.
8. Eliminar o aislar `/proyectos-test` del router de produccion.
9. Dar salida funcional a command palette y acciones rapidas placeholders.
10. Ejecutar y estabilizar pruebas automatizadas reales, incluyendo E2E autenticadas.

9. Veredicto Final

DesignSystemMissionControl no esta "casi terminado" en terminos de produccion, aunque visualmente lo parezca. La lectura mas honesta es esta: la plataforma ya tiene suficiente cuerpo para ser demostrable, operable en entornos controlados y acelerable hacia release, pero todavia no alcanza el umbral de fiabilidad, seguridad y consistencia que justifica llamarla 100% lista.

El porcentaje real de finalizacion estimado en esta auditoria es **73%**. La distancia restante no es tanto de volumen de pantallas, sino de cierre tecnico en autenticacion, Ads, Settings, pruebas y hardening.

Reporte generado localmente a partir de inspeccion del workspace del proyecto. Fuente principal: **DesignSystemMissionControl**. Fecha de emision: 2026-03-19.